

Tutorial1-5习题及答案整理

- Tutorial1-5习题及答案整理

- Tutorial 1 练习题

- 练习1.1: 实现 Fibonacci 函数

- 练习1.2: 计算列表的累乘

- 练习1.3: 实现函数组合 (※)

- 练习1.4: 实现 for_all 函数 (※)

- Tutorial 2 练习题

- 练习2.1: 判断尾递归

- 练习2.2: 判断单元素列表

- 练习2.3: 改写为尾递归

- 练习2.4: 使用 List.map

- 练习2.5: 实现filter函数

- 练习2.6: 统计子序列

- 练习2.7: 实现尾递归求和

- 练习2.8: 数列差分

- 练习2.9: 实现 fold_right

- 练习2.10: 实现 smallbf 解释器

- Tutorial 3 练习题

- 练习 3.1 Calculator

- 练习 3.2 Bank Account

- 练习 3.3 Map

- 练习 3.4 Pokemon Fight

- 练习 3.5 Task

- Tutorial 4 练习题

- 练习 4.1

- 练习 4.2

练习 4.3

Tutorial 5 练习题

练习 5.1 calculator

练习 5.2 SimPL

练习 5.3 Lambda

Tutorial 1 练习题

练习1.1：实现 Fibonacci 函数

- 题目：编写一个递归函数 `fib`，计算第 n 个 Fibonacci 数 F_n ，其定义为： $F_n = 0$ (若 $n=0$), 1 (若 $n=1$), $F_{n-1} + F_{n-2}$ (其他情况)。
- 调用示例：`fib 19` 应该返回 `4181`。
- 要求：
 1. 函数签名：`val fib : int -> int`。
 2. 对于非法输入 ($n < 0$) 的情况要适当处理。

```
1 let rec fib n =  
2   if n < 0 then failwith "n shoule be non-negative"  
3   else if n = 0 then 0  
4   else if n = 1 then 1  
5   else fib (n-1) + fib (n-2)  
6 let result = fib 5;;
```

练习1.2：计算列表的累乘

- 题目：编写一个递归函数 `prod`，计算列表 `[x1; x2; ...; xn]` 的累乘。
- 调用示例：`prod [1; 2; 3; 4; 5]` 应该返回 `120`。
- 要求：
 1. 函数签名：`val prod : int list -> int`。

2. 使用模式匹配处理列表。

```
1 let rec prod lst =  
2   match lst with  
3   | [] -> 1  
4   | head :: tail -> head * myfun tail  
5 let result = prod [1;2;3;4;5];;
```

练习1.3：实现函数组合（※）

- 题目：编写一个函数 `compose`，实现两个函数 `f` 和 `g` 的组合运算 `f ∘ g`，其定义为 $(f \circ g)(x) = f(g(x))$ 。
- 调用示例：

```
1 let double x = x * 2  
2 let add_one x = x + 1  
3 let f = compose double add_one  
4 f 3 (* 应该返回 8, 即 (3+1)*2 *)
```

- 要求：

1. 函数签名： `val compose : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b`。

```
1 let double x = x * 2  
2 let add_two x = x + 2  
3 let compose f1 f2 n = f1(f2 n)  
4 let f = compose double add_two  
5 let result = f 3;;
```

练习1.4：实现 `for_all` 函数（※）

- 题目：编写一个函数 `for_all`，传入一个谓词函数 `p: 'a -> bool` 和一个列表 `lst`，检查列表中的所有元素是否都满足 `p` 指定的条件。
- 调用示例：

```

1 let is_positive x = x > 0
2 for_all is_positive [1; 2; 3] (* true *)
3 for_all is_positive [-1; 2; 3] (* false *)
4 for_all is_positive [] (* true *)

```

- 要求：

1. 函数签名： `val for_all : ('a -> bool) -> 'a list -> bool`。
2. 空列表应返回 `true`。

```

1 let rec for_all p lst =
2     match lst with
3     | [] -> true
4     | head :: tail -> (p head) && (for_all p tail)
5
6 let is_positive x = x > 0
7 let is_even x = x mod 2 = 0
8 let r1 = for_all is_positive [1; 2; 3]
9 let r2 = for_all is_even [2; 4; 7]
10 let r3 = for_all is_positive []

```

Tutorial 2 练习题

练习2.1：判断尾递归

- 题目：给定函数 `let rec is_prime_helper x i = if i == 1 then true else (x mod i <> 0) && (is_prime_helper x (i-1))`。

 1. 这个函数是尾递归吗？
 2. 设计一个实验来判断它是否为尾递归（例如，传入一个非常大的数，观察是否会导致栈溢出）。

练习2.2：判断单元素列表

- 题目：实现 `has_single_elem : 'a list -> bool`，判断列表是否刚好只包含一个元素。

- 调用示例: `has_single_elem []` (* false) `has_single_elem [1]` (true)
`has_single_elem [1; 2]` (false *)

```

1 let has_single_elem l =
2   match l with
3   | [] -> false
4   | _ :: [] -> true
5   | _ -> false
6
7 let r1 =has_single_elem []
8 let r2 =has_single_elem [1]
9 let r3 =has_single_elem [2;3;4]

```

练习2.3：改写为尾递归

- 题目：将 `let rec list_max l = match l with | [] -> failwith "..."/>`

```

1 let rec list_max_iter l m =
2   match l with
3   | [] -> m
4   | x :: xs ->
5     let new_m = max x m in
6     list_max_iter xs new_m

```

练习2.4：使用 List.map

- 题目：用 `List.map` 函数求一个列表所有元素的平方。

```

1 let square_list l = List.map (fun x->x*x) l
2 let result=square_list [1;2;3;4]

```

练习2.5：实现filter函数

- 题目：实现一个函数 `filter : ('a -> bool) -> 'a list -> 'a list`。

```
1 let rec my_filter f l =
2   match l with
3   | [] -> []
4   | h::t ->
5       if f h then
6         h::my_filter f t
7       else
8         my_filter f t
```

练习2.6：统计子序列

- 题目：实现 `count_123 : int list -> int`，返回列表中连续子序列 `1; 2; 3` 出现的次数。
- 调用示例： `count_123 [1; 2; 3; 1; 2; 3; 4]` 应该返回 `2`。

```
1 let rec count_123 l =
2   match l with
3   | [] -> 0
4   | _::[] -> 0
5   | _::_:[] -> 0
6   | h1::h2::h3::t ->
7       if [h1;h2;h3]=[1;2;3] then 1 + count_123 t
8       else count_123 (h2::h3::t)
9
10 let r1= count_123 [1; 2]
11 let r2=count_123 [1;2;3]
12 let r3=count_123 [1;2;4;3]
13 let r4=count_123 [1;2;3;1;4;2;3;1;2;3;4]
```

练习2.7：实现尾递归求和

- 题目：实现尾递归函数 `sum_iter : int list -> int -> int`，对一个可能非常长的列表求和。
- 调用示例： `sum_iter [2;3;4;5] 1` 应该返回 `15`。

```

1 let rec sum_iter l acc =
2   match l with
3   | [] -> acc
4   | h::t -> sum_iter t (acc+h)

```

练习2.8：数列差分

- 题目：实现 `diff : int list -> int list`，求一个整数数列的差分。
- 调用示例：`diff [1; 2; 3; 4; 5]` 应该返回 `[1; 1; 1; 1]`。`diff [1; 2; 4; 1; 1]` 应该返回 `[1; 2; -3; 0]`。

```

1 let rec diff l=
2   match l with
3   | [] -> []
4   | _::[] -> []
5   | h1::h2::t -> h2-h1:: diff (h2::t)
6 let r1=diff[1;2;3;4;5]
7 let r2=diff [1;2;4;1;1]
8 let r3=diff [1;0;1;0;1;0;0]

```

练习2.9：实现 fold_right

- 题目：实现一个函数 `fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`。
- 调用示例：`fold_right (-) [1; 3; 4] 2` 应该返回 `0`（计算过程为 `1 - (3 - (4 - 2))`）。

```

1 let rec fold_right f l acc =
2   match l with
3   | [] -> acc
4   | h :: t -> f h (fold_right f t acc)
5
6 let r1=fold_right(-)[1;3;4]2

```

练习2.10：实现 smallbf 解释器

- 题目：实现函数 `smallbf : int list -> string -> int list`。它接受一个整数列表 `mem` 和一个命令字符串 `cmd`。指针初始指向 `mem` 的第0个元素。`+` 命令将指针处的数字加1；`>` 命令将指针向右移动一位，若到末尾则回到开头。
- 调用示例：`smallbf [0; 0; 0] "++>>+>++"` 应该返回 `[4; 0; 1]`。

```
1 let rec samllbf mem cmd=
2   let len=List.length mem in
3   let rec aux mem cmd ptr=
4     match cmd with
5     | [] -> mem
6     | '+'::t -> let new_mem=List.mapi(fun i x -> if i=ptr then x+1
7 else x) mem in aux new_mem t ptr
8     | '>'::t -> let new_ptr=(ptr+1) mod len in aux mem t new_ptr
9     | _ -> failwith "Invalid command!" in aux mem (List.of_seq
10 (String.to_seq cmd)) 0
11
12   let r1=samllbf [0;0;0] "++>>+>++"
13   let r2=samllbf [3;0;-1;-2] "+>++++>>>>"
14   let r3=samllbf [42] ">>>>>>>+>+>>"
```

Tutorial 3 练习题

练习 3.1 Calculator

目的

- 体会 OCaml 中 Variant 的定义和使用。
- 复习之前的 Tutorial。

题目描述

请使用 OCaml 语言，实现一个简单的算术表达式计算器。

我们使用以下代数数据类型 `expr` 来表示算术表达式，你需要补充乘法和除法的定义：


```
1 | type expr =  
2 | | Const of int | Add of expr * expr | Sub of expr * expr
```

请你编写一个函数 `evaluate`，用于计算给定表达式 `expr` 的值。函数 `evaluate` 应当满足：

```
1 | val evaluate : expr -> int option
```

该函数返回 `Some v`，表示计算出的整数结果 `v`。该函数返回 `None`，表示计算失败，比如除以零的情况。

样例

```
1 | (* 3 + 2 * 5 *)  
2 |  
3 | let expr = Add (Const 3, Mul (Const 2, Const 5))  
4 |  
5 | let result = evaluate expr
```

输出是：

```
1 | (* 3 + 2 * 5 = 13 *)  
2 |  
3 | val result : int option = Some 13
```

提示:使用递归函数和模式匹配来完成这个任务。

```
1 | (* 表达式的递归变体定义 *)  
2 | type expr =  
3 | | Const of int  
4 | | Add of expr * expr  
5 | | Sub of expr * expr  
6 | | Mul of expr * expr  
7 | | Div of expr * expr  
8 |
```

```

9  (* 计算表达式的值
10  - 如果表达式包含除以零的情况，则返回 None
11  - 否则返回 Some 计算结果
12  *)
13  let rec evaluate (expr : expr) : int option =
14      (* 递归计算表达式的值 *)
15      match expr with
16      | Const n -> Some n
17      | Add (e1, e2) -> (
18          match evaluate e1, evaluate e2 with
19          | Some v1, Some v2 -> Some (v1 + v2)
20          | _ -> None
21      )
22      | Sub (e1, e2) -> (
23          match evaluate e1, evaluate e2 with
24          | Some v1, Some v2 -> Some (v1 - v2)
25          | _ -> None
26      )
27      | Mul (e1, e2) -> (
28          match evaluate e1, evaluate e2 with
29          | Some v1, Some v2 -> Some (v1 * v2)
30          | _ -> None
31      )
32      | Div (e1, e2) -> (
33          match evaluate e1, evaluate e2 with
34          | Some v1, Some v2 -> if v2 = 0 then None else Some (v1 /
35  v2)
36          | _ -> None
37      )

```

练习 3.2 Bank Account

目的

- 学习使用 OCaml 中的 mutable 和 ref 。
- 学习使用 OCaml 中的 exception。

题目描述

创建一个银行账户系统，使其具备以下功能：

- 账户创建：每个账户必须包含唯一的用户 ID 和所有者姓名。
- 存款：向账户余额中添加一定金额。如果存款金额为负数，抛出 `InvalidDeposit` 异常。
- 取款：从账户余额中取出一定金额：如果余额足够，正常扣除金额并返回。如果余额不足，抛出 `InsufficientFunds` 异常。如果取款金额为负数，抛出 `InvalidWithdrawal` 异常。
- 记录流水：你应该用一个 `list` 记录下来存取款的历史记录
- 查询余额：获取当前账户余额。

你需要实现一个 `bank_account` 类型。某些值应存储在一个可变字段或引用中，以便更新值。

```
1 type bank_account = {  
2  
3 ...  
4  
5 }
```

你还需要实现四个函数：存款 `deposit` 、取款 `withdraw` 、查询余额 `get_balance` 以及输

出记录 `print_transactions`

```

1  val deposit : bank_account -> int -> unit
2
3  val withdraw : bank_account -> int -> unit
4
5  val get_balance : bank_account -> int
6
7  val print_transactions : bank_account -> unit

```

提示:ref 和 mutable 二选一。使用 try ... with 捕获异常并抛出。你可以设计一个变体 transaction，由存取款两个类型构造器组成。

```

1  (* 事务类型
2     - Deposit: 存款
3     - Withdrawal: 取款
4  *)
5  type transaction =
6      | Deposit of int
7      | Withdrawal of int
8
9  (* 银行账户类型
10     - account_id: 账户ID
11     - owner: 账户持有人
12     - balance: 账户余额
13     - transactions: 交易记录列表
14  *)
15  type bank_account = {
16      account_id : int;
17      owner : string;
18      mutable balance : int;
19      mutable transactions : transaction list;
20  }
21
22  (* 异常 *)
23  exception InvalidDeposit
24  exception InsufficientFunds
25  exception InvalidWithdrawal
26
27  (* 创建新账户

```

```

28     - id: 账户ID
29     - owner: 账户持有人
30     - 返回新创建的银行账户
31 *)
32 let create_account (id : int) (owner : string) : bank_account =
33     { account_id = id; owner; balance = 0; transactions = [] }
34
35 (* 存款操作
36     - account: 银行账户
37     - amount: 存款金额
38     - 如果金额大于0, 则将金额添加到账户余额, 并记录交易
39     - 如果金额为负数, 抛出 InvalidDeposit 异常
40 *)
41 let deposit (account : bank_account) (amount : int) : unit =
42     if amount < 0 then raise InvalidDeposit
43     else if amount > 0 then (
44         account.balance <- account.balance + amount;
45         account.transactions <- Deposit amount ::
account.transactions
46     )
47
48 (* 取款操作
49     - account: 银行账户
50     - amount: 取款金额
51     - 如果金额大于0且账户余额足够, 则从账户余额中扣除金额, 并记录交易
52     - 如果金额为负数, 抛出 InvalidWithdrawal 异常
53     - 如果余额不足, 抛出 InsufficientFunds 异常
54 *)
55 let withdraw (account : bank_account) (amount : int) : unit =
56     if amount < 0 then raise InvalidWithdrawal
57     else if account.balance < amount then raise InsufficientFunds
58     else if amount > 0 then (
59         account.balance <- account.balance - amount;
60         account.transactions <- Withdrawal amount ::
account.transactions
61     )
62
63 (* 获取账户余额
64     - account: 银行账户
65     - 返回账户余额
66 *)
67 let get_balance (account : bank_account) : int =

```

```

68     account.balance
69
70 (* 打印交易记录
71    - account: 银行账户
72    - 打印账户的所有交易记录
73 *)
74 let print_transactions (account : bank_account) : unit =
75     let print_transaction = function
76         | Deposit amt -> Printf.printf "Deposit: %d\n" amt
77         | Withdrawal amt -> Printf.printf "Withdrawal: %d\n" amt
78     in
79     List.iter print_transaction (List.rev account.transactions)

```

练习 3.3 Map

目的

- 掌握 OCaml 中模块的定义。
- 掌握类型变量的使用。

题目描述:映射（Map）是由键值对组成的无序集合，在这个练习中你需要实现一个自己的 Map:

```

1  module type Map = sig
2      (** [( 'k, 'v) t] 是一个映射的类型，它将键 ('k 类型) 绑定到值 ('v 类型)。
3      *)
4      type ('k, 'v) t
5      (** [empty] 表示一个不包含任何键的映射。 *)
6      val empty : ('k, 'v) t
7      (** [insert k v m] 返回一个新的映射，其中 [k] 绑定到 [v]，并且包含 [m]
8      中的所有绑定。
9      如果 [k] 在 [m] 中已经有绑定，则新映射中 [k] 的绑定将被 [v] 替代。 *)
10     val insert : 'k -> 'v -> ('k, 'v) t -> ('k, 'v) t
11     (** [lookup k m] 返回在映射 [m] 中与键 [k] 绑定的值。
12     如果 [k] 在 [m] 中没有绑定，则会引发 [Not_found] 异常。 *)
13     val lookup : 'k -> ('k, 'v) t -> 'v

```

```
12 | (** [bindings m] 返回一个关联列表，该列表包含 [m] 中所有的绑定关系。  
13 | 该列表中的键保证是唯一的。 *)  
14 | val bindings : ('k, 'v) t -> ('k * 'v) list  
15 | end
```

样例1

```
1 | let m = MyMap.empty;;  
2 | let m = MyMap.insert "a" 1 m;;  
3 | let m = MyMap.insert "b" 2 m;;  
4 | let m = MyMap.insert "c" 3 m;;  
5 | MyMap.lookup "b" m;;  
6 | MyMap.lookup "a" m;;
```

```
1 | -: int = 2  
2 | -: int = 1
```

样例2

```
1 | let m = Map.empty;;  
2 | let m = Map.insert "one" 1 m;;  
3 | let m = Map.insert "two" 2 m;;  
4 | let m = Map.insert "three" 3 m;;  
5 | Map.bindings m;;
```

```
1 | [("one", 1); ("two", 2); ("three", 3)]
```

样例3

```
1 | let m = Map.empty;;  
2 | let m = Map.insert "alpha" 42 m;;  
3 | Map.lookup "beta" m;;
```

```
1 | Exception: Not_found.
```

提示：你可以使用 list 来实现 Map 模块。模块的签名仅仅规定了模块向外暴露的项，你可以定义几个辅助函数

```
1 (* 实现来自 Cornell cs3110 *)
2 (*
3   https://cs3110.github.io/textbook/chapters/modules/functional\_data\_structures.html#maps *)
4 module type Map = sig
5   (** [('k, 'v) t] is the type of maps that bind keys of type
6       'k to
7       values of type 'v. *)
8   type ('k, 'v) t
9
10  (** [empty] does not bind any keys. *)
11  val empty : ('k, 'v) t
12
13  (** [insert k v m] is the map that binds [k] to [v], and also
14      contains
15      all the bindings of [m]. If [k] was already bound in [m],
16      that old
17      binding is superseded by the binding to [v] in the returned
18      map. *)
19  val insert : 'k -> 'v -> ('k, 'v) t -> ('k, 'v) t
20
21  (** [lookup k m] is the value bound to [k] in [m]. Raises:
22      [Not_found] if [k]
23      is not bound in [m]. *)
24  val lookup : 'k -> ('k, 'v) t -> 'v
25
26  (** [bindings m] is an association list containing the same
27      bindings as [m].
28      The keys in the list are guaranteed to be unique. *)
29  val bindings : ('k, 'v) t -> ('k * 'v) list
30 end
31
32 module AssocListMap : Map = struct
33   (** The list [(k1, v1); ...; (kn, vn)] binds key [ki] to value
34       [vi].
```



```

28 |         If a key appears more than once in the list, it is bound to
    | the
29 |         the left-most occurrence in the list. *)
30 | type ('k, 'v) t = ('k * 'v) list
31 | let empty = []
32 | let insert k v m = (k, v) :: m
33 | let lookup k m = List.assoc k m
34 | let keys m = List.(m |> map fst |> sort_uniq Stdlib.compare)
35 | let bindings m = m |> keys |> List.map (fun k -> (k, lookup k
    | m))
36 | end

```

练习 3.4 Pokemon Fight

目的

- 掌握 Functor 的用法。

题目描述： 你需要模拟一个简单的宝可梦（Pokémon）战斗系统：

1. 定义 Pokemon 模块类型，其中包含属性：宝可梦的名称、生命值、攻击力、受到攻击时减少生命值：

```

1 | module type Pokemon = sig、
2 |   (* 名称 *)
3 |   val name : string
4 |   (* 生命值 *)
5 |   val hp : int ref
6 |   (* 攻击力 *)
7 |   val attack : int
8 |   (* 收到攻击时减少生命值，减少的数量应该是对方宝可梦的攻击力 *)
9 |   val take_damage : int -> unit
10| end

```

2. 实现具体的宝可梦模块，它们应该符合 Pokemon 模块类型，比如：

```

1 module Pikachu : Pokemon = struct
2   let name = "Pikachu"
3   ...
4 end

```

3.定义 Battle Functor，它接受 两个符合 Pokemon 模块类型的模块，并提供战斗逻辑：

- `fight : unit -> unit` —— 两个宝可梦轮流攻击，直到其中一个生命值为 0。
- `print_winner : unit -> unit` —— 输出胜利者的名字。

```

1 module Battle : functor (P1 : Pokemon) (P2 : Pokemon) -> sig
2   val fight : unit -> unit
3   val print_winner : unit -> unit
4 end

```

样例：

```

1 (* 假设你已经定义好了 Pikaku 模块和 Charmander 模块 *)
2 module PikachuVsCharmander = Battle (Pikachu) (Charmander)
3 let () = PikachuVsCharmander.fight();
  PikachuVsCharmander.print_winner ();;

```

```

1 Pikachu wins!

```

```

1 (* 宝可梦类
2   - name: 宝可梦名称
3   - hp: 宝可梦生命值
4   - attack: 宝可梦攻击力
5   - take_damage: 宝可梦受到伤害时调用的方法
6 *)
7 module type Pokemon = sig
8   val name : string
9   val hp : int ref

```

```
10     val attack : int
11     val take_damage : int -> unit
12 end
13
14 (* 皮卡丘类
15     - name: 皮卡丘名称
16     - hp: 皮卡丘生命值
17     - attack: 皮卡丘攻击力
18     - take_damage: 皮卡丘受到伤害时调用的方法
19 *)
20 module Pikachu : Pokemon = struct
21     let name = "Pikachu"
22     let hp = ref 100
23     let attack = 20
24     let take_damage dmg = hp := max 0 (!hp - dmg)
25 end
26
27 (* 小火龙类
28     - name: 小火龙名称
29     - hp: 小火龙生命值
30     - attack: 小火龙攻击力
31     - take_damage: 小火龙受到伤害时调用的方法
32 *)
33 module Charmander : Pokemon = struct
34     let name = "Charmander"
35     let hp = ref 80
36     let attack = 25
37     let take_damage dmg = hp := max 0 (!hp - dmg)
38 end
39
40 (* 宝可梦战斗模块
41     - P1: 宝可梦1
42     - P2: 宝可梦2
43     - fight: 宝可梦之间的战斗函数
44     - print_winner: 打印获胜者的函数
45 *)
46 module Battle (P1 : Pokemon) (P2 : Pokemon) = struct
47     let rec fight () =
48         P2.take_damage P1.attack;
49         if !(P2.hp) = 0 then ()
50         else (
51             P1.take_damage P2.attack;
```

```
52     if !(P1.hp) > 0 then fight ()
53 )
54
55 let print_winner () =
56     if !(P1.hp) > 0 then
57         Printf.printf "%s wins!\n" P1.name
58     else
59         Printf.printf "%s wins!\n" P2.name
60 end
61
```

练习 3.5 Task

目的

- 培养面向对象编程的思想和能力。
- 掌握 OCaml 中 Class 的用法。

题目描述

你需要设计一个任务管理系统，支持管理不同类型的任务（普通任务、定时任务、优先任务），并实现以下功能：

1. 任务类型：
 - 普通任务：仅包含标题和描述。
 - 定时任务：在普通任务的基础上还包含开始时间和结束时间。
 - 优先任务：在普通任务的基础上还包含优先级（如 1–5，数字越大优先级越高）。
2. 任务管理器：开发一个任务管理器管理所有任务。
3. 任务操作：
 - 添加任务：向任务管理器添加任务。
 - 列出任务：以字符串格式显示所有任务信息。
 - 排序任务：先按优先级排序（优先任务 > 其他任务）。再按开始时间排序（仅适用于定时任务）。
 - 状态管理：任务可以标记为完成或未完成。

提示 :这道题目不对实现方式作出限制，你可以自由的设计类和继承关系。尝试运用封装、继承和多态。

Tutorial 4 练习题

练习 4.1

使用ocamllex和ocamlyacc实现对四则运算的词法和语法分析，使用dune构建项目。

1.验证dune环境

```
1 | dune --version
```

以下是问题备忘：vscode使用dune可能遇到的一个问题是，.ml文件莫名其妙爆红，可能的办法是 在构建的时候使用以下命令：

```
1 | dune build @ocaml-index
2 | 如果显示没有ocaml-index的话就运行下面命令安装后重启vscode，再运行上面的构建命令：
3 | opam install ocaml-index
```

2.创建项目目录，创建dune配置文件

```
1 | mkdir my_project && cd my_project
2 | mkdir lib
3 | touch dune dune-project lib/dune
```

3.创建词法和语法分析器文件: lexer.mll, parser.mly, ast.ml

```

1 (* 创建 lib/lexer.mll *)
2 {
3 open Parser
4 }
5 rule read = parse
6 | [' ' '\t' '\n'] { read lexbuf } (* 跳过空白符 *)
7 | ['0'-'9']+ as num { INT (int_of_string num) }
8 | '+' { PLUS }
9 | '-' { MINUS }
10 | '*' { TIMES }
11 | '/' { DIV }
12 | eof { EOF }
13 | _ { failwith "Unknown character" }

```

```

1 // 创建 lib/parser.mly
2 %{
3 open Ast
4 %}
5 %token <int> INT
6 %token PLUS MINUS TIMES DIV EOF
7 %start main
8 %type <Ast.expr> main
9 %%
10 main:
11 expr EOF { $1 }
12 ;
13 expr:
14 | term PLUS expr { Add($1, $3) }
15 | term MINUS expr { Sub($1, $3) }
16 | term { $1 }
17 ;
18 term:
19 | factor TIMES term { Mul($1, $3) }
20 | factor DIV term { Div($1, $3) }
21 | factor { $1 }
22 ;
23 factor:
24 | INT { Num($1) }
25 ;

```

```
1 (* 创建 lib/ast.ml *)
2 type expr =
3 | Num of int
4 | Add of expr * expr
5 | Sub of expr * expr
6 | Mul of expr * expr
7 | Div of expr * expr
```

4.创建main.ml

```
1 (* 创建 main.ml *)
2 open Lex_project
3 open Ast
4 let rec string_of_expr = function
5 | Num n -> string_of_int n
6 | Add (e1, e2) -> Printf.sprintf "(%s + %s)" (string_of_expr e1)
7 (string_of_expr e2)
8 | Sub (e1, e2) -> Printf.sprintf "(%s - %s)" (string_of_expr e1)
9 (string_of_expr e2)
10 | Mul (e1, e2) -> Printf.sprintf "(%s * %s)" (string_of_expr e1)
11 (string_of_expr e2)
12 | Div (e1, e2) -> Printf.sprintf "(%s / %s)" (string_of_expr e1)
13 (string_of_expr e2)
14 let () =
15 let lexbuf = Lexing.from_channel stdin in
16 let ast = Parser.main Lexer.read lexbuf in
17 Printf.printf "\nAST: %s\n" (string_of_expr ast)
```

5.写dune配置

```
1 | ; lib/dune
2 | (library
3 | (name lex_project)
4 | (modules lexer ast parser))
5 | ; dune
6 | (executable
7 | (name main)
8 | (libraries lex_project))
9 | ; dune-project
10 | (lang dune 3.0)
```

6.使用ocamllex和ocamlyacc生成词法和语法文件，dune编译并运行

```
1 | ocamllex lib/lexer.mll
2 | ocamlyacc -b lib/parser lib/parser.mly
3 | dune build
4 | ./_build/default/main.exe
```

运行后输入四则运算如 $1 + 2 * 3 - 4$ ，按下ctrl + D作为EOF结束符，看看生成的AST是否正确保证了运算优先级.

练习 4.2

现有一个文法 $G[E]$ 如下，请你为其实现递归下降语法分析，实现parse函数，检验一个char list是否合法。

```
1 | G[E]:
2 | E->aAB | bB
3 | A->aB | b
4 | B->bA | a
```

注意： 你不需要设计AST数据结构进行存储，只需要判断一个字符串是否可以由该文法产生，输出“Success”或“Failed”。思考 $E \rightarrow aAB$ 的处理方式

示例输入：


```
1 | let test_input = ['a'; 'a'; 'a'; 'a']
```

示例输出：

```
1 | Success
```

```
1 (* 解析E *)
2 let rec parseE input =
3   match input with
4   | 'a'::rest -> parseB (parseA rest)
5   | 'b'::rest -> parseB rest
6   | _ -> raise (Failure "error in parseE")
7
8 (* 解析A *)
9 and parseA input =
10  match input with
11  | 'a'::rest -> parseB rest
12  | 'b'::rest -> rest
13  | _ -> raise (Failure "error in parseA")
14
15 (* 解析B *)
16 and parseB input =
17  match input with
18  | 'b'::rest -> parseA rest
19  | 'a'::rest -> rest
20  | _ -> raise (Failure "error in parseB")
21
22 let parse input =
23   try
24     let remaining = parseE input in
25     if remaining = [] then "Success"
26     else "Failed: Unexpected leftover input"
27   with
28   | Failure msg -> "Failed: " ^ msg
29
30 let test_input = ['a'; 'a'; 'a'; 'a']
31
```

```
32 let () = print_endline (parse test_input)
```

练习 4.3

下面是一个简单的函数式语言的语法，请你任选两种语法分析方法之一：

- ocaml yacc
- 递归下降语法分析

实现一个词法和语法分析器，能够为该语法的任一合法句子生成所对应的抽象语法树 AST。

1	<code>t ::=</code>	terms
2	<code>true</code>	constant true
3	<code>false</code>	constant false
4	<code>if t then t else t</code>	conditional
5	<code>0</code>	constant zero
6	<code>succ t</code>	successor
7	<code>pred t</code>	predecessor
8	<code>iszero t</code>	zero test

参考代码： <https://www.cis.upenn.edu/~bcpierce/tapl/checkers/arith/>

参考书籍：《Types and Programming Languages》

示例输入：

```
1 if false then 0 else succ 0
2 iszero pred succ 0
```

注意：

- 你可以自定义输入输出方式，如参考练习1一样使用命令行输入
- 你可以实现自己的AST打印函数，以检查生成效果
- 建议预留扩展词法语法分析的接口

3-yacc

- main.ml

```
1 open Lex_project
2 open Ast
3
4 let rec string_of_term = function
5   | True -> "true"
6   | False -> "false"
7   | Zero -> "0"
8   | Succ t -> Printf.sprintf "succ(%s)" (string_of_term t)
9   | Pred t -> Printf.sprintf "pred(%s)" (string_of_term t)
10  | IsZero t -> Printf.sprintf "iszero(%s)" (string_of_term t)
11  | If (cond, t1, t2) ->
12      Printf.sprintf "(if %s then %s else %s)"
13          (string_of_term cond)
14          (string_of_term t1)
15          (string_of_term t2)
16
17 let () =
18   try
19     let lexbuf = Lexing.from_channel stdin in
20     let ast = Parser.main Lexer.main lexbuf in
21     Printf.printf "\nAST: %s\n" (string_of_term ast)
22   with
23   | Failure msg ->
24     Printf.eprintf "Error: %s\n" msg
```

- ast.ml

```

1 type term =
2   | True
3   | False
4   | Zero
5   | Succ of term
6   | Pred of term
7   | IsZero of term
8   | If of term * term * term

```

- lever.mll

```

1 {
2   open Parser
3 }
4
5 rule main = parse
6   | [' ' '\t' '\n']+ { main lexbuf }
7
8   | "true"           { TRUE }
9   | "false"          { FALSE }
10  | "if"              { IF }
11  | "then"            { THEN }
12  | "else"            { ELSE }
13  | "0"               { ZERO }
14  | "succ"            { SUCC }
15  | "pred"            { PRED }
16  | "iszero"          { ISZERO }
17
18  | eof               { EOF }
19
20  | _                 { failwith "Unrecognized token" }

```

- parser.mly

```

1 %{
2   open Ast
3 %}
4
5 %token TRUE FALSE IF THEN ELSE ZERO SUCC PRED ISZERO EOF
6 %type <Ast.term> main

```

```

7  %start main
8
9  %%
10
11  main:
12      term EOF { $1 }
13
14  ;
15
16  term:
17      TRUE          { True }
18  | FALSE          { False }
19  | ZERO           { Zero }
20  | SUCC term      { Succ($2) }
21  | PRED term      { Pred($2) }
22  | ISZERO term    { IsZero($2) }
23  | IF term THEN term ELSE term { If($2, $4, $6) }
24  ;

```

- dune

```

1  (lang dune 3.0)
2
3  (executable
4    (name main)
5    (libraries lex_project))
6
7
8  (library
9    (name lex_project)
10   (modules lexer ast parser))

```

3–recursive descent

- main.ml

```

1  open Lex_project
2  open Ast

```

```

3
4 let rec string_of_term = function
5   | True -> "true"
6   | False -> "false"
7   | Zero -> "0"
8   | Succ t -> Printf.sprintf "succ(%s)" (string_of_term t)
9   | Pred t -> Printf.sprintf "pred(%s)" (string_of_term t)
10  | IsZero t -> Printf.sprintf "iszero(%s)" (string_of_term t)
11  | If (cond, t1, t2) ->
12      Printf.sprintf "(if %s then %s else %s)"
13          (string_of_term cond)
14          (string_of_term t1)
15          (string_of_term t2)
16
17 let () =
18   try
19     let lexbuf = Lexing.from_channel stdin in
20     let ast = Parser.parse lexbuf in
21     Printf.printf "\nAST: %s\n" (string_of_term ast)
22   with
23   | Failure msg ->
24     Printf.eprintf "Error: %s\n" msg

```

- ast.ml

```

1 type term =
2   | True
3   | False
4   | Zero
5   | Succ of term
6   | Pred of term
7   | IsZero of term
8   | If of term * term * term

```

- lever.mll

```

1 {
2 open Token
3 }
4
5 rule main = parse

```

```

6 | [' ' '\t' '\n']+ { main lexbuf }
7
8 | "true"           { TRUE }
9 | "false"          { FALSE }
10 | "if"             { IF }
11 | "then"           { THEN }
12 | "else"           { ELSE }
13 | "0"              { ZERO }
14 | "succ"           { SUCC }
15 | "pred"           { PRED }
16 | "iszero"         { ISZERO }
17
18 | eof              { EOF }
19
20 | _                { failwith "Unrecognized token" }

```

- token.ml

```

1 | type token =
2 |   | TRUE
3 |   | FALSE
4 |   | IF
5 |   | THEN
6 |   | ELSE
7 |   | ZERO
8 |   | SUCC
9 |   | PRED
10 |  | ISZERO
11 |  | EOF

```

- parser.ml

```

1 | open Ast
2 | open Token
3
4 | let next_token lexbuf = Lexer.main lexbuf
5
6 | let string_of_token = function
7 |   | TRUE -> "TRUE"
8 |   | FALSE -> "FALSE"
9 |   | ZERO -> "ZERO"

```

```

10 | SUCC -> "SUCC"
11 | PRED -> "PRED"
12 | ISZERO -> "ISZERO"
13 | IF -> "IF"
14 | THEN -> "THEN"
15 | ELSE -> "ELSE"
16 | EOF -> "EOF"
17
18 let expect token lexbuf =
19   let current = next_token lexbuf in
20   if current <> token then
21     failwith (Printf.sprintf "\nUnexpected token: %s"
(string_of_token token))
22
23 let rec parseTerm lexbuf =
24   match next_token lexbuf with
25   | TRUE -> True
26   | FALSE -> False
27   | ZERO -> Zero
28   | SUCC ->
29     let t = parseTerm lexbuf in
30     Succ t
31   | PRED ->
32     let t = parseTerm lexbuf in
33     Pred t
34   | ISZERO ->
35     let t = parseTerm lexbuf in
36     IsZero t
37   | IF ->
38     let cond = parseTerm lexbuf in
39     expect THEN lexbuf;
40     let t1 = parseTerm lexbuf in
41     expect ELSE lexbuf;
42     let t2 = parseTerm lexbuf in
43     If (cond, t1, t2)
44   | token ->
45     failwith (Printf.sprintf "\nUnexpected token: %s"
(string_of_token token))
46
47 let parse lexbuf =
48   let ast = parseTerm lexbuf in
49   match next_token lexbuf with

```



```

50 |     | EOF -> ast
51 |     | token ->
52 |         failwith (Printf.sprintf "\nUnexpected token %s after
    parsing" (string_of_token token))

```

- dune

```

1 | (lang dune 3.0)
2 |
3 | (executable
4 |   (name main)
5 |   (libraries lex_project))
6 |
7 | (library
8 |   (name lex_project)
9 |   (modules token lexer ast parser))

```

Tutorial 5 练习题

练习 5.1 calculator

- main.ml

```

1 | open Mathlib
2 | open Ast
3 |
4 | let rec string_of_expr (e : expr) : string =
5 |   match e with
6 |   | Int n -> Printf.sprintf "Int %d" n
7 |   | Binop (binop, e1, e2) ->
8 |       let binop_str =
9 |         match binop with
10 |         | Add -> "Add"
11 |         | Mul -> "Mul"
12 |         | Sub -> "Sub"
13 |         | Div -> "Div"
14 |       in
15 |       Printf.sprintf "Binop (%s, %s, %s)" binop_str (string_of_expr
    e1) (string_of_expr e2)

```

```

16
17
18 let parse s : expr =
19     let lexbuf = Lexing.from_string s in
20     let ast = Parser.main Lexer.read lexbuf in
21     ast
22
23
24 (* check if an expression is a value (i.e., fully evaluated) *)
25 let is_value : expr -> bool = function
26     | Int _ -> true
27     | Binop _ -> false
28
29
30 (* takes a single step of evaluation of [e] *)
31 let rec step : expr -> expr = function
32     | Int _ -> failwith "Does not step on a number"
33
34     (* No need for further stepping if both sides are already
35     values *)
36     | Binop (binop, e1, e2) when is_value e1 && is_value e2 ->
37         step_binop binop e1 e2
38
39     (* Evaluate the right side of the binop if the left side is a
40     value *)
41     | Binop (binop, e1, e2) when is_value e1 -> Binop (binop, e1,
42         step e2)
43
44     (* Leftmost step for binop *)
45     | Binop (binop, e1, e2) -> Binop (binop, step e1, e2)
46
47
48 (* implement the primitive operation [v1 binop v2].
49 Requires: [v1] and [v2] are both values. *)
50 and step_binop binop v1 v2 = match binop, v1, v2 with
51     | Add, Int a, Int b -> Int (a + b)
52     | Sub, Int a, Int b -> Int (a - b)
53     | Mul, Int a, Int b -> Int (a * b)
54     | Div, Int a, Int b when b <> 0 -> Int (a / b)
55     | Div, Int _, Int 0 -> failwith "Division by zero"
56     | _ -> failwith "Operator and operand type mismatch"

```

```

55
56 (* fully evaluate [e] to a value [v] *)
57 let rec eval (e : expr) : expr =
58     if is_value e then e else
59         e |> step |> eval
60
61
62 (* interpret [s] by lexing -> parsing -> evaluating and
63    converting the result to a string *)
64 let interp (s : string) : string =
65     s |> parse |> eval |> string_of_expr
66
67 let rec eval_big (e : expr) : expr = match e with
68     | Int _ -> e
69     | Binop (binop, e1, e2) -> eval_bop binop e1 e2
70
71 and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2
72 with
73 | Add, Int a, Int b -> Int (a + b)
74 | Sub, Int a, Int b -> Int (a - b)
75 | Mul, Int a, Int b -> Int (a * b)
76 | Div, Int a, Int b when b <> 0 -> Int (a / b)
77 | Div, Int _, Int 0 -> failwith "Division by zero"
78 | _ -> failwith "Operator and operand type mismatch"
79
80 let interp_big (s : string) : string =
81     s |> parse |> eval_big |> string_of_expr
82
83 let () =
84     let filename = "test/math_test2.in" in
85     let in_channel = open_in filename in
86     let file_content = really_input_string in_channel
87     (in_channel_length in_channel) in
88     close_in in_channel;
89
90     let res = interp file_content in
91     Printf.printf "Result of interpreting %s:\n%s\n\n" filename
92     res;
93
94     let res = interp_big file_content in

```

```

92   Printf.printf "Result of interpreting %s with big-step
    model:\n%s\n\n" filename res;
93
94   let ast = parse file_content in
95   Printf.printf "AST: %s\n" (string_of_expr ast);
96

```

- ast.ml

```

1  type binop =
2      | Add
3      | Sub
4      | Mul
5      | Div
6
7  type expr =
8      | Int of int
9      | Binop of binop * expr * expr
10

```

- lexer.ml

```

1  {
2      open Parser
3  }
4
5  (* 规则定义 *)
6  rule read = parse
7      | [' ' '\t' '\n'] { read lexbuf }
8      | ['0'-'9']+ as num { INT (int_of_string num) }
9      | '+' { PLUS }
10     | '-' { MINUS }
11     | '*' { TIMES }
12     | '/' { DIV }
13     | '(' { LPAREN }
14     | ')' { RPAREN }
15     | eof { EOF }
16     | _ { failwith "Invalid character" }

```

- parser.mly

```

1  %{
2      open Ast
3  %}
4
5  %token <int> INT
6  %token PLUS MINUS TIMES DIV EOF
7  %token LPAREN RPAREN
8
9  (*Left associated, and also the order that TIMES DIV is behind
10 PLUS MINUS means that TIMES and DIV have higher precedence.*)
11 %left PLUS MINUS
12 %left TIMES DIV
13
14 %start main
15 %type <Ast.expr> main
16 %%
17
18 main:
19     expr EOF { $1 }
20 ;
21
22 expr:
23     | INT { Int $1 }
24     | expr TIMES expr { Binop (Mul, $1, $3) }
25     | expr DIV expr { Binop (Div, $1, $3) }
26     | expr PLUS expr { Binop (Add, $1, $3) }
27     | expr MINUS expr { Binop (Sub, $1, $3) }
28     | LPAREN expr RPAREN { $2 }
29 ;

```

- dune-project

```

1  (lang dune 3.17)
2
3  (name interpreter_project)
4
5  (generate_opam_files true)
6
7  (source
8    (github AugustineYang/OCaml-Interpreter))
9

```

```

10 (authors "Yang Yuanzhao <yang_yuanzhao@whu.edu.cn>")
11
12 (license LICENSE)
13
14 (package
15   (name interpreter_project)
16   (synopsis "A short synopsis")
17   (description "A longer description")
18   (depends ocaml)
19   (tags
20     ("add topics" "to describe" your project)))
21
22 ; See the complete stanza docs at
   https://dune.readthedocs.io/en/stable/reference/dune-
   project/index.html
23

```

- lib/dune

```

1 (library
2   (name mathlib)
3   (modules parser lexer ast))
4
5 (ocamllex lexer)
6 (ocamlyacc parser)

```

- bin/dune

```

1 (executable
2   (public_name interpreter_project)
3   (name main)
4   (modules main)
5   (libraries mathlib)
6   (flags (:standard -w -32-27-26-39)))

```

练习 5.2 SimPL

- main.ml

```

1 open Interpreterlib
2 open Ast
3
4 let rec string_of_expr (e : expr) : string =
5     match e with
6     | Int n -> Printf.sprintf "Int %d" n
7     | Var id -> Printf.sprintf "Var %s" id
8     | Bool b ->
9         let b_str =
10             match b with
11             | true -> "true"
12             | false -> "false"
13         in
14         Printf.sprintf "Bool %s" b_str
15     | Binop (binop, e1, e2) ->
16         let binop_str =
17             match binop with
18             | Add -> "Add"
19             | Mul -> "Mul"
20             | Sub -> "Sub"
21             | Div -> "Div"
22             | Leq -> "Leq"
23         in
24         Printf.sprintf "Binop (%s, %s, %s)" binop_str
25         (string_of_expr e1) (string_of_expr e2)
26     | Let (var, e1, e2) -> Printf.sprintf "Let (%s, %s, %s)" var
27         (string_of_expr e1) (string_of_expr e2)
28     | If (e1, e2, e3) -> Printf.sprintf "If (%s, %s, %s)"
29         (string_of_expr e1) (string_of_expr e2) (string_of_expr e3)
30
31 let parse s : expr =
32     let lexbuf = Lexing.from_string s in
33     let ast = Parser.main Lexer.read lexbuf in
34     ast
35
36 (* check if an expression is a value (i.e., fully evaluated) *)
37 let is_value : expr -> bool = function
38     | Int _ | Bool _ -> true
39     | Var _ | Binop _ | If _ | Let _ -> false

```

```

40
41 (** [subst e v x] is [e] with [v] substituted for [x], that
42     is, [e{v/x}]. *)
43 let rec subst e v x = match e with
44   | Var y -> if x = y then v else e
45   | Bool _ -> e
46   | Int _ -> e
47   | Binop (binop, e1, e2) -> Binop (binop, subst e1 v x, subst
48     e2 v x)
49   | If (e1, e2, e3) -> If (subst e1 v x, subst e2 v x, subst e3
50     v x)
51   | Let (y, e1, e2) ->
52     let e1' = subst e1 v x in
53     if x = y
54     then Let (y, e1', e2)
55     else Let (y, e1', subst e2 v x)
56
57 (* takes a single step of evaluation of [e] *)
58 let rec step : expr -> expr = function
59   | Int _ | Bool _ -> failwith "Does not step on a number"
60   | Var _ -> failwith "Unbound variable"
61
62   (* No need for further stepping if both sides are already
63   values *)
64   | Binop (binop, e1, e2) when is_value e1 && is_value e2 ->
65     step_binop binop e1 e2
66
67   (* Evaluate the right side of the binop if the left side is a
68   value *)
69   | Binop (binop, e1, e2) when is_value e1 -> Binop (binop, e1,
70     step e2)
71
72   (* Leftmost step for binop *)
73   | Binop (binop, e1, e2) -> Binop (binop, step e1, e2)
74
75   | If (Bool true, e2, _) -> e2
76   | If (Bool false, _, e3) -> e3
77
78   | If (Int _, _, _) -> failwith "Condition must be a boolean"

```



```

77
78 | If (e1, e2, e3) -> If (step e1, e2, e3)
79
80 | Let (x, e1, e2) when is_value e1 -> subst e2 e1 x
81
82 | Let (x, e1, e2) -> Let (x, step e1, e2)
83
84
85 (* implement the primitive operation [v1 binop v2].
86    Requires: [v1] and [v2] are both values. *)
87 and step_binop binop v1 v2 = match binop, v1, v2 with
88 | Add, Int a, Int b -> Int (a + b)
89 | Sub, Int a, Int b -> Int (a - b)
90 | Mul, Int a, Int b -> Int (a * b)
91 | Div, Int a, Int b when b <> 0 -> Int (a / b)
92 | Div, Int _, Int 0 -> failwith "Division by zero"
93 | Leq, Int a, Int b -> Bool (a <= b)
94 | _ -> failwith "Operator and operand type mismatch"
95
96
97 (* fully evaluate [e] to a value [v] *)
98 let rec eval (e : expr) : expr =
99     if is_value e then e else
100     e |> step |> eval
101
102
103 (* interpret [s] by lexing -> parsing -> evaluating and
104    converting the result to a string *)
105 let interp (s : string) : string =
106     s |> parse |> eval |> string_of_expr
107
108 let rec eval_big (e : expr) : expr = match e with
109 | Int _ | Bool _ -> e
110 | Var _ -> failwith "Unbound variable"
111 | Binop (binop, e1, e2) -> eval_bop binop e1 e2
112 | Let (x, e1, e2) -> subst e2 (eval_big e1) x |> eval_big
113 | If (e1, e2, e3) -> eval_if e1 e2 e3
114
115 and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2
116 with
117 | Add, Int a, Int b -> Int (a + b)

```

```

117 | Sub, Int a, Int b -> Int (a - b)
118 | Mul, Int a, Int b -> Int (a * b)
119 | Div, Int a, Int b when b <> 0 -> Int (a / b)
120 | Div, Int _, Int 0 -> failwith "Division by zero"
121 | Leq, Int a, Int b -> Bool (a <= b)
122 | _ -> failwith "Operator and operand type mismatch"
123
124 and eval_if e1 e2 e3 = match eval_big e1 with
125   | Bool true -> eval_big e2
126   | Bool false -> eval_big e3
127   | _ -> failwith "Condition must be a boolean"
128
129
130 let interp_big (s : string) : string =
131   s |> parse |> eval_big |> string_of_expr
132
133 let () =
134   let filename = "test/simpl_test1.in" in
135   (* let filename = "test/simpl_test2.in" in *)
136   let in_channel = open_in filename in
137   let file_content = really_input_string in_channel
138   (in_channel_length in_channel) in
139   close_in in_channel;
140
141   let res = interp file_content in
142   Printf.printf "Result of interpreting %s:\n%s\n\n" filename
143   res;
144
145   let res = interp_big file_content in
146   Printf.printf "Result of interpreting %s with big-step
147   model:\n%s\n\n" filename res;
148
149   let ast = parse file_content in
150   Printf.printf "AST: %s\n" (string_of_expr ast);
151

```

- ast.ml

```

1 type binop =
2   | Add
3   | Sub
4   | Mul

```

```

5   | Div
6   | Leq
7
8   type expr =
9     | Int of int
10    | Var of string
11    | Bool of bool
12    | Binop of binop * expr * expr
13    | Let of string * expr * expr
14    | If of expr * expr * expr
15

```

- lexer.ml

```

1   {
2     open Parser
3   }
4
5   rule read = parse
6     | [' ' '\t' '\n'] { read lexbuf }
7     | '+' { PLUS }
8     | '-' { MINUS }
9     | '*' { TIMES }
10    | '/' { DIV }
11    | '(' { LPAREN }
12    | ')' { RPAREN }
13    | "<=" { LEQ }
14    | "true" { TRUE }
15    | "false" { FALSE }
16    | "let" { LET }
17    | "=" { EQUALS }
18    | "in" { IN }
19    | "if" { IF }
20    | "then" { THEN }
21    | "else" { ELSE }
22    | ['0'-'9']+ as num { INT (int_of_string num) }
23    | ['a'-'z' 'A'-'Z']+ as id { ID id }
24    | eof { EOF }
25    | _ { failwith "Invalid character" }

```

- parser.mly

```

1  %{
2      open Ast
3  %}
4
5  %token <int> INT
6  %token <string> ID
7  %token PLUS MINUS TIMES DIV EOF
8  %token LPAREN RPAREN
9  %token LEQ
10 %token TRUE FALSE
11 %token LET EQUALS IN
12 %token IF THEN ELSE
13
14 %nonassoc IN
15 %nonassoc ELSE
16 %left LEQ
17 %left PLUS MINUS
18 %left TIMES DIV
19
20 %start main
21 %type <Ast.expr> main
22 %%
23
24 main:
25     expr EOF { $1 }
26 ;
27
28 expr:
29     | INT { Int $1 }
30     | ID { Var $1 }
31     | TRUE { Bool true }
32     | FALSE { Bool false}
33     | expr LEQ expr { Binop (Leq, $1, $3) }
34     | expr TIMES expr { Binop (Mul, $1, $3) }
35     | expr DIV expr { Binop (Div, $1, $3) }
36     | expr PLUS expr { Binop (Add, $1, $3) }
37     | expr MINUS expr { Binop (Sub, $1, $3) }
38     | LET ID EQUALS expr IN expr { Let ($2, $4, $6) }
39     | IF expr THEN expr ELSE expr { If ($2, $4, $6) }
40     | LPAREN expr RPAREN { $2 }
41 ;

```

- dune-project

```
1 (lang dune 3.17)
2
3 (name interpreter_project)
4
5 (generate_opam_files true)
6
7 (source
8   (github AugustineYang/OCaml-Interpreter))
9
10 (authors "Yang Yuanzhao <yang_yuanzhao@whu.edu.cn>")
11
12 (license LICENSE)
13
14 (package
15   (name interpreter_project)
16   (synopsis "A short synopsis")
17   (description "A longer description")
18   (depends ocaml)
19   (tags
20     ("add topics" "to describe" your project)))
21
22 ; See the complete stanza docs at
23 ; https://dune.readthedocs.io/en/stable/reference/dune-
24 ; project/index.html
```

- lib/dune

```
1 (library
2   (name interpreterlib)
3   (modules parser lexer ast))
4
5 (ocamllex lexer)
6 (ocamlyacc parser)
```

- bin/dune

```

1 (executable
2   (public_name interpreter_project)
3   (name main)
4   (modules main)
5   (libraries interpreterlib)
6   (flags (:standard -w -32-27-26-39-8)))

```

练习 5.3 Lambda

- main.ml

```

1 open Interpreterlib
2 open Ast
3
4 let rec string_of_expr (e : expr) : string =
5   match e with
6   | Int n -> Printf.sprintf "Int %d" n
7   | Var id -> Printf.sprintf "Var %s" id
8   | Bool b ->
9     let b_str =
10       match b with
11       | true -> "true"
12       | false -> "false"
13     in
14     Printf.sprintf "Bool %s" b_str
15   | Binop (binop, e1, e2) ->
16     let binop_str =
17       match binop with
18       | Add -> "Add"
19       | Mul -> "Mul"
20       | Sub -> "Sub"
21       | Div -> "Div"
22       | Leq -> "Leq"
23     in
24     Printf.sprintf "Binop (%s, %s, %s)" binop_str
25     (string_of_expr e1) (string_of_expr e2)
26   | Let (var, e1, e2) -> Printf.sprintf "Let (%s, %s, %s)" var
27     (string_of_expr e1) (string_of_expr e2)

```

```

26   | If (e1, e2, e3) -> Printf.sprintf "If (%s, %s, %s)"
(string_of_expr e1) (string_of_expr e2) (string_of_expr e3)
27   | Func (var, e) -> Printf.sprintf "Func (%s, %s)" var
(string_of_expr e)
28   | App (e1, e2) -> Printf.sprintf "App (%s, %s)"
(string_of_expr e1) (string_of_expr e2)
29
30
31 let parse s : expr =
32   let lexbuf = Lexing.from_string s in
33   let ast = Parser.main Lexer.read lexbuf in
34   ast
35
36
37 (* check if an expression is a value (i.e., fully evaluated) *)
38 let is_value : expr -> bool = function
39   | Int _ | Bool _ | Func _ -> true
40   | Var _ | Binop _ | If _ | Let _ | App _ -> false
41
42 module VarSet = Set.Make(String)
43 let singleton = VarSet.singleton
44 let union = VarSet.union
45 let diff = VarSet.diff
46 let mem = VarSet.mem
47
48 (** [fv e] is a set-like list of the free variables of [e]. *)
49 let rec fv : expr -> VarSet.t = function
50   | Var x -> singleton x
51   | App (e1, e2) -> union (fv e1) (fv e2)
52   | Func (x, e) -> diff (fv e) (singleton x)
53
54 (** [gensym ()] is a fresh variable name. *)
55 let gensym =
56   let counter = ref 0 in
57   fun () ->
58     incr counter; "$x" ^ string_of_int !counter
59
60 (** [replace e y x] is [e] with the name [x] replaced
61     by the name [y] anywhere it occurs. *)
62 let rec replace e y x = match e with
63   | Var z -> if z = x then Var y else e
64   | App (e1, e2) -> App (replace e1 y x, replace e2 y x)

```

```

65 | Func (z, e') -> Func ((if z = x then y else z), replace e' y
66 x)
67 (** [subst e v x] is [e] with [v] substituted for [x], that
68     is, [e{v/x}]. *)
69 let rec subst e v x = match e with
70 | Var y -> if x = y then v else e
71 | Bool _ -> e
72 | Int _ -> e
73 | Binop (binop, e1, e2) -> Binop (binop, subst e1 v x, subst
74 e2 v x)
75 | If (e1, e2, e3) -> If (subst e1 v x, subst e2 v x, subst e3
76 v x)
77 | Let (y, e1, e2) ->
78   let e1' = subst e1 v x in
79   if x = y
80   then Let (y, e1', e2)
81   else Let (y, e1', subst e2 v x)
82 | App (e1, e2) -> App (subst e1 v x, subst e2 v x)
83 | Func (y, e') ->
84   if x = y then e
85   else if not (mem y (fv v)) then Func (y, subst e' v x)
86   else
87     let fresh = gensym () in
88     let new_body = replace e' y fresh in
89     Func (fresh, subst new_body v x)
90
91 (* takes a single step of evaluation of [e] *)
92 let rec step : expr -> expr = function
93 | Int _ | Bool _ -> failwith "Does not step on a number"
94
95 | Var _ -> failwith "Unbound variable"
96
97 (* No need for further stepping if both sides are already
98 values *)
99 | Binop (binop, e1, e2) when is_value e1 && is_value e2 ->
100   step_binop binop e1 e2
101
102 (* Evaluate the right side of the binop if the left side is a
103 value *)

```



```

101 | Binop (binop, e1, e2) when is_value e1 -> Binop (binop, e1,
step e2)
102
103 (* Leftmost step for binop *)
104 | Binop (binop, e1, e2) -> Binop (binop, step e1, e2)
105
106 | If (Bool true, e2, _) -> e2
107
108 | If (Bool false, _, e3) -> e3
109
110 | If (Int _, _, _) -> failwith "Condition must be a boolean"
111
112 | If (e1, e2, e3) -> If (step e1, e2, e3)
113
114 | Let (x, e1, e2) when is_value e1 -> subst e2 e1 x
115
116 | Let (x, e1, e2) -> Let (x, step e1, e2)
117
118
119 (* implement the primitive operation [v1 binop v2].
120    Requires: [v1] and [v2] are both values. *)
121 and step_binop binop v1 v2 = match binop, v1, v2 with
122 | Add, Int a, Int b -> Int (a + b)
123 | Sub, Int a, Int b -> Int (a - b)
124 | Mul, Int a, Int b -> Int (a * b)
125 | Div, Int a, Int b when b <> 0 -> Int (a / b)
126 | Div, Int _, Int 0 -> failwith "Division by zero"
127 | Leq, Int a, Int b -> Bool (a <= b)
128 | _ -> failwith "Operator and operand type mismatch"
129
130
131 (* fully evaluate [e] to a value [v] *)
132 let rec eval (e : expr) : expr =
133   if is_value e then e else
134     e |> step |> eval
135
136
137 (* interpret [s] by lexing -> parsing -> evaluating and
converting the result to a string *)
138 let interp (s : string) : string =
139   s |> parse |> eval |> string_of_expr
140

```

```

141
142 (* CBV stands for call-by-value, and CBN stands for call-by-
    name.
143     If strategy is set to be CBV, the call-by-value strategy is
    adopted.
144     And vice versa.
145
146     Warning 37 should be disabled for this statement.*/)
147 type eval_strategy = CBV | CBN
148 let strategy = CBV
149
150 let rec eval_big (e : expr) : expr = match e with
151   | Int _ | Bool _ -> e
152   | Var _ -> failwith "Unbound variable"
153   | Binop (binop, e1, e2) -> eval_bop binop e1 e2
154   | Let (x, e1, e2) -> subst e2 (eval_big e1) x |> eval_big
155   | If (e1, e2, e3) -> eval_if e1 e2 e3
156   | App (e1, e2) -> eval_app e1 e2
157   | Func _ -> e
158
159 and eval_bop binop e1 e2 = match binop, eval_big e1, eval_big e2
    with
160   | Add, Int a, Int b -> Int (a + b)
161   | Sub, Int a, Int b -> Int (a - b)
162   | Mul, Int a, Int b -> Int (a * b)
163   | Div, Int a, Int b when b <> 0 -> Int (a / b)
164   | Div, Int _, Int 0 -> failwith "Division by zero"
165   | Leq, Int a, Int b -> Bool (a <= b)
166   | _ -> failwith "Operator and operand type mismatch"
167
168 and eval_if e1 e2 e3 = match eval_big e1 with
169   | Bool true -> eval_big e2
170   | Bool false -> eval_big e3
171   | _ -> failwith "Condition must be a boolean"
172
173
174 (** [eval_app e1 e2] is the [e] such that [e1 e2 ==> e]. *)
175 and eval_app e1 e2 = match eval_big e1 with
176   | Func (x, e) ->
177     let e2' =
178       match strategy with
179       | CBV -> eval_big e2

```

```

180     | CBN -> e2
181     in subst e e2' x l> eval_big
182     | _ -> failwith "Cannot apply non-function"
183
184 let interp_big (s : string) : string =
185     s l> parse l> eval_big l> string_of_expr
186
187 let () =
188     (* let filename = "test/lambda_test1.in" in *)
189     (* let filename = "test/lambda_test2.in" in *)
190     (* let filename = "test/lambda_test3.in" in *)
191     (* let filename = "test/lambda_test4.in" in *)
192     let filename = "test/lambda_test5.in" in
193     let in_channel = open_in filename in
194     let file_content = really_input_string in_channel
195     (in_channel_length in_channel) in
196     close_in in_channel;
197
198     (* let res = interp file_content in
199     Printf.printf "Result of interpreting %s:\n%s\n\n" filename
200     res; *)
201
202     let res = interp_big file_content in
203     Printf.printf "Result of interpreting %s with big-step
204     model:\n%s\n\n" filename res;
205
206     let ast = parse file_content in
207     Printf.printf "AST: %s\n" (string_of_expr ast);
208

```

- ast.ml

```

1 type binop =
2     | Add
3     | Sub
4     | Mul
5     | Div
6     | Leq
7
8 type expr =
9     | Int of int
10    | Var of string

```

```

11 | Bool of bool
12 | Binop of binop * expr * expr
13 | Let of string * expr * expr
14 | If of expr * expr * expr
15 | Func of string * expr
16 | App of expr * expr
17

```

- lexer.mll

```

1 | {
2 |     open Parser
3 | }
4
5 | rule read = parse
6 |   | [' ' '\t' '\n'] { read lexbuf }
7 |   | '+' { PLUS }
8 |   | '-' { MINUS }
9 |   | '*' { TIMES }
10 |  | '/' { DIV }
11 |  | '(' { LPAREN }
12 |  | ')' { RPAREN }
13 |  | "<=" { LEQ }
14 |  | "true" { TRUE }
15 |  | "false" { FALSE }
16 |  | "let" { LET }
17 |  | "=" { EQUALS }
18 |  | "in" { IN }
19 |  | "if" { IF }
20 |  | "then" { THEN }
21 |  | "else" { ELSE }
22 |  | "->" { ARROW }
23 |  | "fun" { FUNC }
24 |  | ['0'-'9']+ as num { INT (int_of_string num) }
25 |  | ['a'-'z' 'A'-'Z']+ as id { ID id }
26 |  | eof { EOF }
27 |  | _ { failwith "Invalid character" }

```

- parser.mly

```

1 | %{

```

```

2      open Ast
3
4      (** [make_apply e [e1; e2; ...]] makes the application
5          [e e1 e2 ...]). Requires: the list argument is non-empty. *)
6      let rec make_apply e = function
7          | [] -> failwith "precondition violated"
8          | [e'] -> App (e, e')
9          | h :: (_ :: _) as t -> make_apply (App (e, h)) t
10     %}
11
12     %token <int> INT
13     %token <string> ID
14     %token PLUS MINUS TIMES DIV EOF
15     %token LPAREN RPAREN
16     %token LEQ
17     %token TRUE FALSE
18     %token LET EQUALS IN
19     %token IF THEN ELSE
20     %token FUNC ARROW
21
22     %nonassoc IN
23     %nonassoc ELSE
24     %left LEQ
25     %left PLUS MINUS
26     %left TIMES DIV
27
28     %start main
29     %type <Ast.expr> main
30     %%
31
32     main:
33         expr EOF { $1 }
34     ;
35
36     expr:
37         | simpl_expr { $1 }
38         | simpl_expr simpl_expr+ { make_apply $1 $2 }
39         | FUNC ID ARROW expr { Func ($2, $4) }
40     ;
41
42     simpl_expr:
43         | INT { Int $1 }

```

```

44 | ID { Var $1 }
45 | TRUE { Bool true }
46 | FALSE { Bool false}
47 | simpl_expr LEQ simpl_expr { Binop (Leq, $1, $3) }
48 | simpl_expr TIMES simpl_expr { Binop (Mul, $1, $3) }
49 | simpl_expr DIV simpl_expr { Binop (Div, $1, $3) }
50 | simpl_expr PLUS simpl_expr { Binop (Add, $1, $3) }
51 | simpl_expr MINUS simpl_expr { Binop (Sub, $1, $3) }
52 | LET ID EQUALS simpl_expr IN simpl_expr { Let ($2, $4, $6)
}
53 | IF simpl_expr THEN simpl_expr ELSE simpl_expr { If ($2, $4,
$6) }
54 | LPAREN expr RPAREN { $2 }
55 ;

```

- dune-project

```

1 (lang dune 3.17)
2
3 (name interpreter_project)
4
5 (generate_opam_files true)
6
7 (using menhir 3.0)
8
9 (source
10  (github AugustineYang/OCaml-Interpreter))
11
12 (authors "Yang Yuanzhao <yang_yuanzhao@whu.edu.cn>")
13
14 (license LICENSE)
15
16 (package
17  (name interpreter_project)
18  (synopsis "A short synopsis")
19  (description "A longer description")
20  (depends ocaml)
21  (tags
22   ("add topics" "to describe" your project)))
23

```

```
24 | ; See the complete stanza docs at  
    | https://dune.readthedocs.io/en/stable/reference/dune-  
    | project/index.html
```

```
25
```

- lib/dune

```
1 | (library  
2 |   (name interpreterlib)  
3 |   (modules parser lexer ast))  
4 |  
5 | (ocamllex lexer)  
6 | (menhir  
7 |   (modules parser))
```

- bin/dune

```
1 | (executable  
2 |   (public_name interpreter_project)  
3 |   (name main)  
4 |   (modules main)  
5 |   (libraries interpreterlib)  
6 |   (flags (:standard -w -32-27-26-39-8-37)))
```